

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010

OPERATING SYSTEM

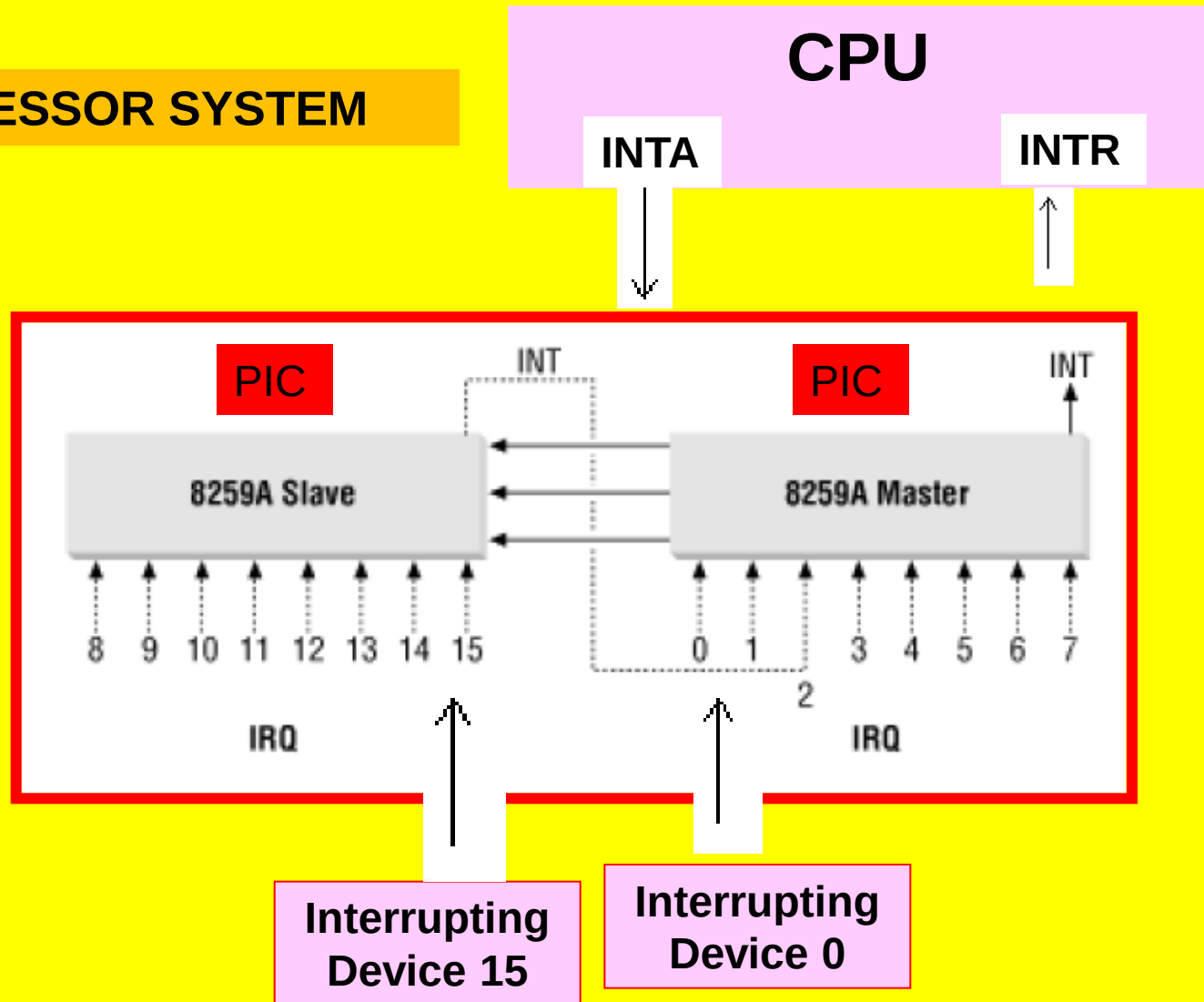
WEEK 5 LECTURE 2

INTERRUPTS & EXCEPTIONS

WISH YOU ALL THE BEST

IRQs and INTERRUPTS

UNIPROCESSOR SYSTEM



INTERRUPTS

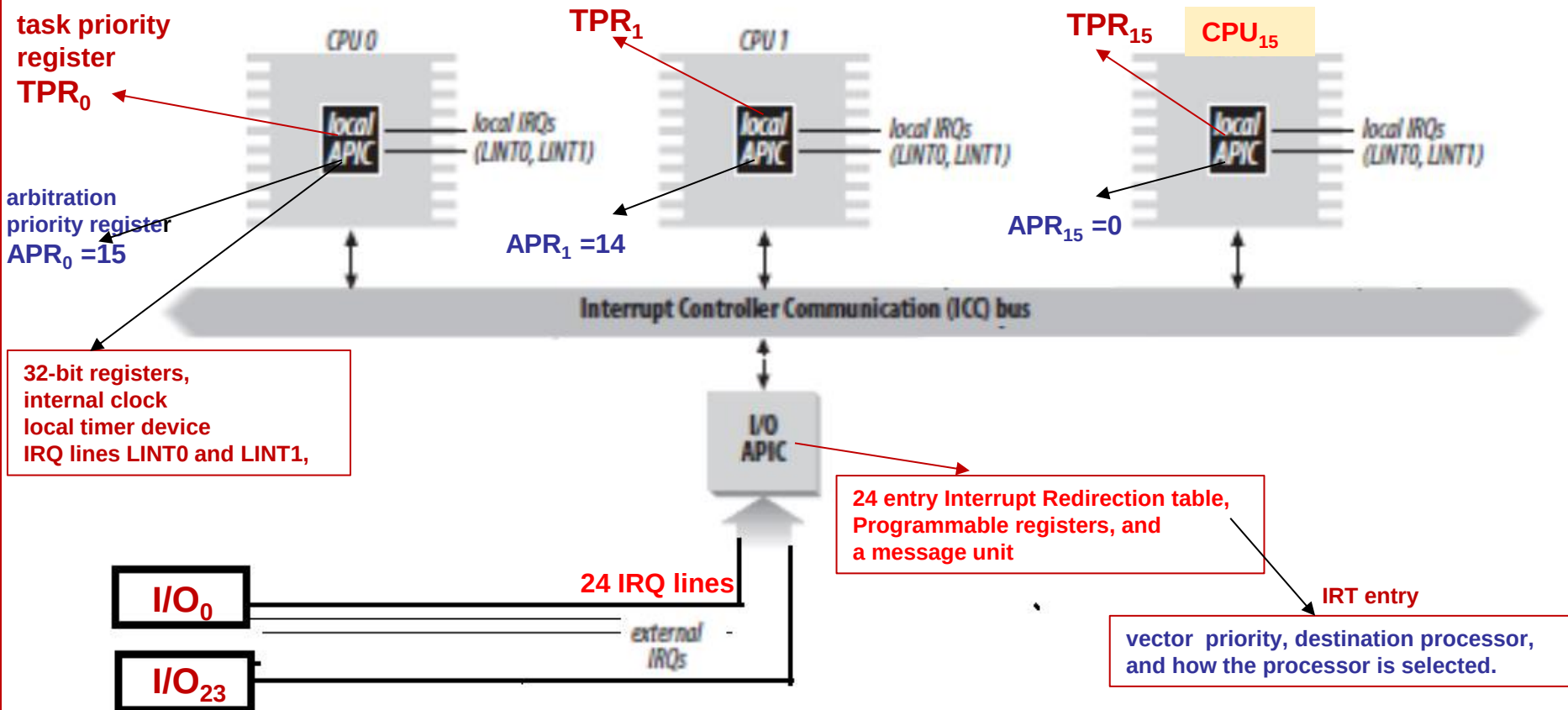
- * Figure illustrates a typical connection "in cascade" of two Intel 8259A PICs that can handle up to 15 different IRQ input lines.
- * Notice that the INT output line of the second PIC is connected to the IRQ2 pin of the first PIC: a signal on that line denotes the fact that an IRQ signal on any one of the lines IRQ8-IRQ15 has occurred.
- * The number of available IRQ lines is thus traditionally limited to 15; however, more recent PIC chips are able to handle many more input lines.
- * Each IRQ line can be selectively disabled.
- * Thus, the PIC can be programmed to disable IRQs.
- * That is, the PIC can be told to stop issuing interrupts that refer to a given IRQ line or vice versa to enable them.
- * Disabled interrupts are not lost; the PIC sends them to the CPU as soon as they are enabled again.
- * This feature is used by most interrupt handlers, since it allows them to process IRQs of the same type serially.
- * Selective enabling/disabling of IRQs is not the same as global masking/unmasking of maskable interrupts.
- * When the IF flag of the eflags register is clear, any maskable interrupt issued by the PIC is simply ignored by the CPU.
- * The cli and sti Assembly instructions, respectively, clear and set that flag.

ADVANCED PROGRAMMABLE INTERRUPT CONTROLLER (APIC)

- * If the system includes a single CPU (Uniprocessor System), the output line of the master PIC can be connected in a straightforward way to the INTR pin the CPU.
- * However, if the system includes two or more CPUs, this approach is no longer valid and more sophisticated PICs are needed.
- * Being able to deliver interrupts to each CPU in the system is crucial for fully exploiting the parallelism of the SMP architecture.
- * For that reason, Intel introduced starting with Pentium III a new component designated as the **I/O Advanced Programmable Interrupt Controller (I/O APIC)**.
- * This chip is the advanced version of the old 8259A Programmable Interrupt controller; to support old operating systems, recent motherboards include both types of chip.
- * Moreover, **all current 80 x 86 microprocessors include a local APIC.**
- * **Each local APIC has 32-bit registers, an internal clock; a local timer device; and two additional IRQ lines, LINT0 and LINT1, reserved for local APIC interrupts.**
- * **All local APICs are connected to an external I/O APIC, giving rise to a multi-APIC system.**

MULTI-APIC SYSTEM

- * Figure illustrates in schematic way the structure of a multi-APIC system.
- * An APIC bus connects the "frontend" I/O APIC to the local APIC to the local APICs.
- * The IRQ lines coming from the devices are connected to the I/O APIC, which therefore acts as router with respect to the local APICs.
- * In the motherboards of the Pentium III and earlier processors, the APIC bus was a serial three-line bus; starting with the Pentium 4, the APIC bus is implemented by means of the system bus.



ADVANCED PROGRAMMABLE INTERRUPT CONTROLLER (APIC)

- ★ The I/O APIC consists of a set of 24 IRQ lines, a 24 entry Interrupt Redirection table, programmable registers, and a message unit for sending and receiving APIC messages over the APIC bus.
- ★ Unlike IRQ pins of the 8259A, interrupt priority is not related to pin number: each entry in the Redirection Table can be individually programmed to indicate vector and priority, the destination processor, and how the processor is selected.
- ★ The information in the redirection table is used to translate each external IRQ signal into a message to one or more local APIC units via the APIC bus.

ADVANCED PROGRAMMABLE INTERRUPT CONTROLLER (APIC)

- ★ Interrupt requests coming from external hardware devices can be distributed among the available CPUs in two ways:
 - ❑ **Static distribution**
 - ❑ **Dynamic distribution**
- ❑ **Static distribution**
 - ★ The IRQ signal is delivered to the local APICs listed in the corresponding Redirection Table entry.
 - ★ The interrupt is delivered to one specific CPU, to a subset of CPUs, or to all CPUs at once (broadcast mode).

ADVANCED PROGRAMMABLE INTERRUPT CONTROLLER (APIC)

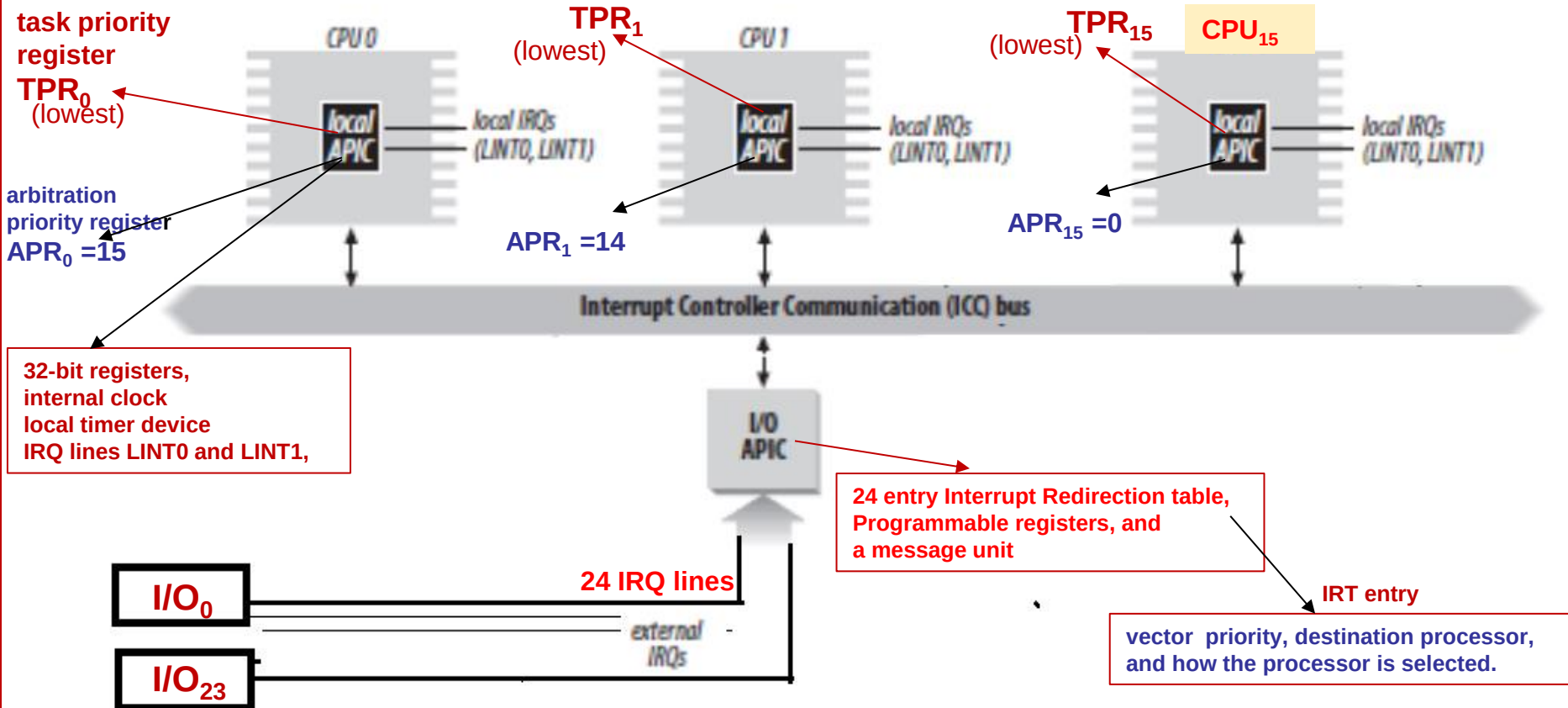
❑ Dynamic distribution

- * The IRQ signal is delivered to the local APIC of the processor that is executing the process with the lowest priority.
- * Every local APIC has a programmable task priority register (TPR), which is used to compute the priority of the currently running process.
- * Intel expects this register to be modified in an operating system kernel by each process switch.
- * If two or more CPUs share the lowest priority, the load is distributed between them using a technique called arbitration.
- * Each CPU is assigned a different arbitration priority ranging from 0 (lowest) to 15 (highest) in the arbitration priority register of the local APIC.
- * Every time an interrupt is delivered to a CPU, its corresponding arbitration priority is automatically set to 0, while the arbitration priority of any other CPU is increased.
- * When the arbitration priority register becomes greater than 15, it is set to the previous arbitration priority of the winning CPU increased by 1.
- * Therefore, interrupts are distributed in a round-robin fashion among CPUs with the same task priority.

MULTI-APIC SYSTEM

□ DYNAMIC DISTRIBUTION

- * Every local APIC has a programmable task priority register (TPR), which is used to compute the priority of the currently running process
- * If two or more CPUs share the lowest priority, the load is distributed between them using a technique called arbitration.
- * Each CPU is assigned a different arbitration priority ranging from 0 (lowest) to 15 (highest) in the arbitration priority register of the local APIC. Every time an interrupt is delivered to a CPU, its corresponding arbitration priority is automatically set to 0, while the arbitration priority of any other CPU is increased
- * Therefore, interrupts are distributed in a round-robin fashion among CPUs with the same task priority.



❑ Interprocessor interrupts

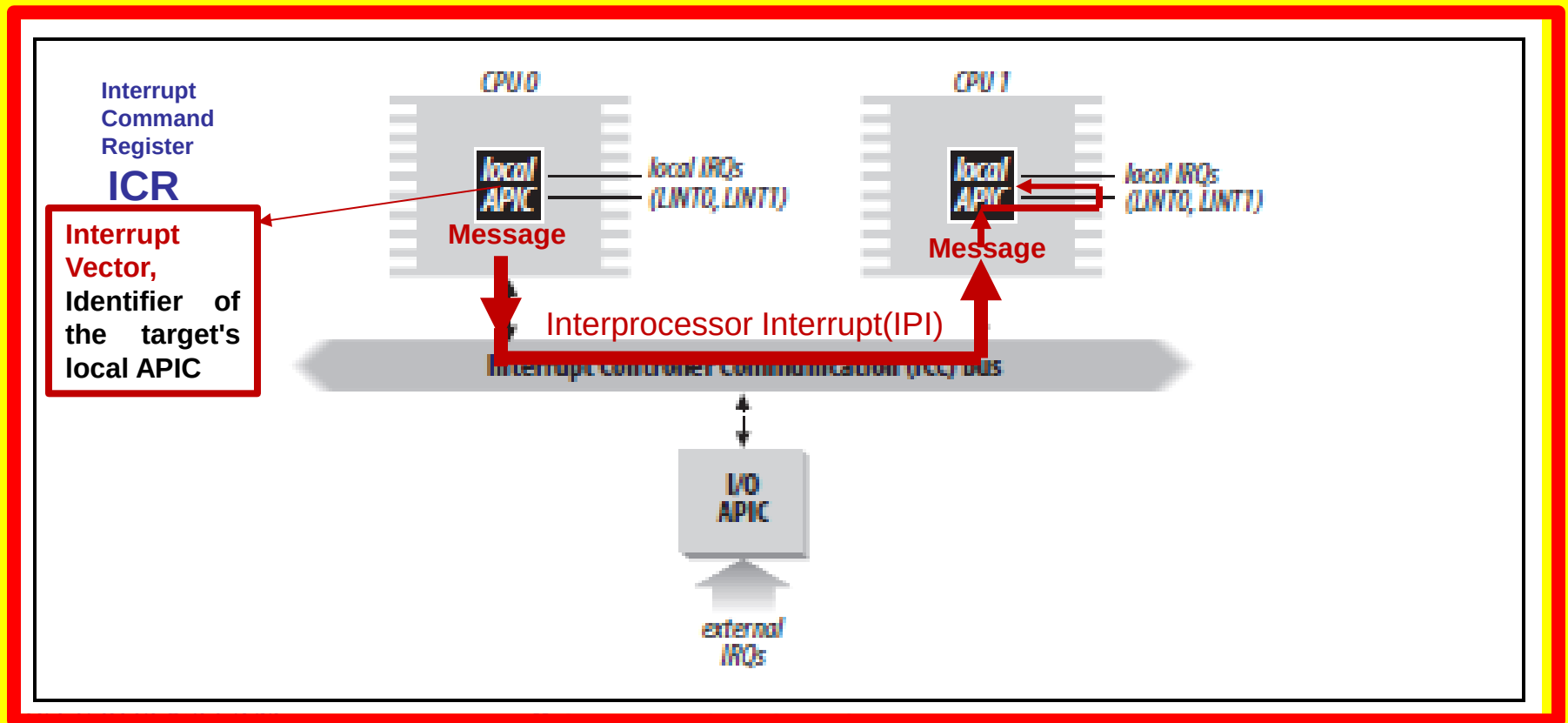
- ★ Besides distributing interrupts among processors, the multi-PIC system allows CPUs to **generate interprocessor interrupts**.
- ★ When a CPU wishes to send an interrupt to another CPU, it stores the interrupt vector and the identifier of the target's local APIC in the Interrupt Command Register (ICR) of its own local APIC.
- ★ A message is then sent via the APIC bus to the target's local APIC, which therefore issues a corresponding interrupt to its own CPU.
- ★ Interprocessor interrupts (in short, IPIs) are a crucial component of the SMP architecture.
- ★ They are actively used by Linux to exchange messages among CPUs



ADVANCED PROGRAMMABLE INTERRUPT CONTROLLER (APIC)

★ Interprocessor interrupts.

- The multi-PIC system allows CPUs to **generate interprocessor interrupts**.
- When a CPU wishes to send an interrupt to another CPU, it stores the interrupt vector and the identifier of the target's local APIC in the Interrupt Command Register (ICR) of its own local APIC.
- A message is then sent via the APIC bus to the target's local APIC, which therefore issues a corresponding interrupt to its own CPU.



ADVANCED PROGRAMMABLE INTERRUPT CONTROLLER (APIC)

- ★ Many of the current uniprocessor systems include an I/O APIC chip, which may be configured in two distinct ways:
 - . As a standard 8259A-style external PIC connected to the CPU. The local APIC is disabled and the two LINT0 and LINT1 local IRQ lines are configured, respectively, as the INTR and NMI pins.
 - . As a standard external I/O APIC. The local APIC is enabled, and all external interrupts are received through the I/O APIC.



EXCEPTIONS

❑ Exceptions

- * The Intel 80x86 microprocessors issue roughly 20 different exceptions. The kernel must provide a dedicated exception handler for each exception type. For some exceptions, the CPU control unit also generates a hardware error code and pushes it in the Kernel Mode stack before starting the exception handler

0 - "Divide error" (fault)

- * Raised when a program tries to **divide by 0**.

1- "Debug" (trap or fault)

- * Raised when the T flag of eflags is set (quite **useful to implement step-by-step execution** of a debugged program) or when the address of an instruction or operand falls within the range of an active debug register

2 - Not used

- * Reserved for **nonmaskable interrupts (those that use the NMI pin)**.

3 - "Breakpoint" (trap)

Caused by an **int3 (breakpoint) instruction** (usually inserted by a debugger).

EXCEPTIONS

4 - "Overflow" (trap)

- ★ An int 4 (check for overflow) instruction has been executed when the OF (overflow) flag of eflags is set.

5 - "Bounds check" (fault)

- ★ A bound (check on address bound) instruction has been executed with the operand outside of the valid address bounds.

6 - "Invalid opcode" (fault)

- ★ The CPU execution unit has detected an invalid opcode (the part of the machine instruction that determines the operation performed).

7 - "Device not available" (fault)

- ★ An ESCAPE or MMX instruction has been executed with the TS flag of cr0 set.

EXCEPTIONS

8 - "Double fault" (abort)

- ★ Normally, when the CPU detects an exception while trying to call the handler for a prior exception, the two exceptions can be handled serially. In a few cases, however, the processor cannot handle them serially, hence it raises this exception.

9 - "Coprocessor segment overrun" (abort)

- ★ Problems with the external mathematical coprocessor (applies only to old 80386 microprocessors).

10 - "Invalid TSS" (fault)

- ★ The CPU has attempted a context switch to a process having an invalid Task State Segment

EXCEPTIONS

11 - "Segment not present" (fault)

- ★ A reference was made to a segment not present in memory (one in which the Segment-Present flag of the Segment Descriptor was cleared).

12 - "Stack segment" (fault)

- ★ The instruction attempted to exceed the stack segment limit, or the segment identified by ss is not present in memory.

13 - "General protection" (fault)

- ★ One of the protection rules in the protected mode of the Intel 80x86 has been violated.

14 - "Page fault" (fault)

- ★ The addressed page is not present in memory, the corresponding page table entry is null, or a violation of the paging protection mechanism has occurred.

EXCEPTIONS

15 - Reserved by Intel

16 - "Floating point error" (fault)

- ★ The floating point unit integrated into the CPU chip has signaled an error condition, such as numeric overflow or division by 0.

17 - "Alignment check" (fault)

- ★ The address of an operand is not correctly aligned (for instance, the address of a long integer is not a multiple of 4).

18 – “Machine check” (abort)

- ★ A machine-check mechanism has detected a CPU or bus error

19 – “SIMD floating point exception” (fault)

- ★ The SSE or SSE2 unit integrated in the CPU chip has signalled an error condition on a floating-point operation

20 to 31

- ★ These values are reserved by Intel for future development.

EXCEPTIONS

Table . Signals Sent by the Exception Handlers

#	Exception	Exception Handler	Signal
0	"Divide error"	divide_error()	SIGFPE
1	"Debug"	debug()	SIGTRAP
2	NMI	nmi()	None
3	"Breakpoint"	int3()	SIGTRAP
4	"Overflow"	overflow()	SIGSEGV
5	"Bounds check"	bounds()	SIGSEGV
6	"Invalid opcode"	invalid_op()	SIGILL
7	"Device not available"	device_not_available()	SIGSEGV
8	"Double fault"	double_fault()	SIGSEGV
9	"Coprocessor segment overrun"	coprocessor_segment_overrun()	SIGFPE
10	"Invalid TSS"	invalid_tss()	SIGSEGV
11	"Segment not present"	segment_not_present()	SIGBUS
12	"Stack exception"	stack_segment()	SIGBUS
13	"General protection"	general_protection()	SIGSEGV
14	"Page fault"	page_fault()	SIGSEGV
15	Intel reserved	None	None
16	"Floating point error"	coprocessor_error()	SIGFPE
17	"Alignment check"	alignment_check()	SIGSEGV

18. "Machine check"

machine_check()

None

19. SIMD floating point

simd_coprocessor_error()

SIGFPE

SOC 3010

OPERATING SYSTEM

EXERCISE

QUESTIONS

INTERRUPTS &

EXCEPTIONS

INTERRUPTS & EXCEPTIONS

□ Questions

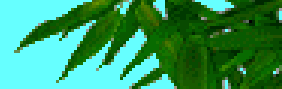
- * Define Interrupt
- * Distinguish between Asynchronous and Synchronous Interrupts
- * Distinguish between Interrupts and Exceptions.
- * How are interrupts caused ?
- * How are exceptions caused ?
- * State the key difference between Interrupt handling and process switching.
- * An interrupt breaks the execution of instructions and diverts its execution to
 - a) Interrupt service routine b) Counter word register
 - c) Execution unit d) Control unit
- * If an interrupt is generated from outside the CPU then it is an
 - a) asynchronous b) synchronous c) unhandleable e) handleable
- * If the interrupt is generated by the execution of an interrupt instruction then it is
 - a) asynchronous b) synchronous c) unhandleable e) handleable
- * Example of an asynchronous interrupt is
 - a) divide by zero interrupt b) keyboard interrupt
 - c) overflow interrupt d) breakpoint interrupt
- * Example of a synchronous interrupt is
 - a) divide by zero interrupt b) keyboard interrupt
 - c) timer interrupt d) type2 interrupt

INTERRUPTS & EXCEPTIONS

❑ Questions

- * NMI stands for
 - a) Non Maskable Interrupt b) Non Multiple Interrupt c) Non Movable interrupt
 - d) None of the above
- * The Programmable interrupt controller is required to
 - a) handle one interrupt request
 - b) handle one or more interrupt requests at a time
 - c) handle one or more interrupt requests with a delay
 - d) handle no interrupt request
- * The interrupt for which the processor has highest priority among all the external interrupts is
 - a) keyboard interrupt b) TRAP c) NMI d) INT
- * The interrupt for which the processor has highest priority among all the internal interrupts is
 - a) keyboard interrupt b) TRAP c) NMI d) INT
- * What are the two types of hardware interrupts?
- * Distinguish between Maskable and Non-maskable Interrupts
- * What are the two types of Exceptions?
- * Explain the following :
 - a) Processor-detected Exceptions b) Programmed Exceptions
- * What are the three groups of Processor Detected Exceptions
- * What are Faults, Traps and Aborts
- * In exception, the saved value of eip is the address of the instruction that caused the exception, and hence that instruction can be resumed when the exception handler terminates.
- * In exception, the saved value of eip is the address of the instruction that should be executed after the one that caused the exception.

INTERRUPTS & EXCEPTIONS



□ Questions

- * Give two examples of fault exceptions.
- * Give two examples of trap exceptions
- * Give two examples of abort exceptions
- * What are Programmed Exceptions (Software Interrupts)?
- * How are the software interrupts caused?
- * What are the two common uses of Software Interrupts?
- * Explain what is meant by interrupt vector.
- * Vectors ranging from 0 to 31 correspond to
- * Vectors ranging from 32 to 47 are assigned to
- * Vector 128 or 0x80 corresponds to
- * What happens when an int 0x80 assembly instruction is executed by a process in user mode.
- * What are the two interrupt pins on the processor.
- * What are the three pins on the processor to handle interrupts? Indicate the purpose of each pin.
- * Describe the functions of the following pins on the processor:
 - a) INTR b) NMI c) INTA
- * The interrupt request on INTR pin can be disabled by resetting the following flag in the processor status register:
 - a. DF b. ZF c. OF d. IF
- * What is 8259 PIC? How many interrupting devices one 8259 PIC chip can support?
- * Which flag of the processor is used to disable or enable maskable Interrupts?
- * What are the intel assembly language instructions to set and clear the IF flag in a program?
- * Say TRUE or FALSE - Interrupt request on NMI pin on the processor can be disabled by resetting the IF flag of the processor status register.....
- * State the purpose of TF flag in the processor.

INTERRUPTS & EXCEPTIONS



□ Questions

- * Intel's default vector associated with IRQ_n line is
a. $n + 8$ b. $n + 16$ c. $n + 32$ d. $n + 64$
- * Write the full form of I/O APIC.
- * List all the components of the I/O APIC
- * State the operations performed by I/O APIC.
- * Say TRUE or FALSE – Each core in the processor has a separate local APIC.....
- * Name the two interrupt lines on local APIC.
- * Draw the structure of a multi-APIC system.
- * Number of Interrupt Request IRQ lines on I/O APIC is
a. 8 b. 16 c. 24 d. 32
- * Explain the purpose of ICC bus.
- * What is the purpose of a 24 entry Interrupt Redirection Table in an I/O APIC?
- * Describe the two ways by which interrupt requests coming from external hardware devices can be distributed among the available CPUs.
- * Explain static distribution of interrupt requests among the available CPUs.
- * Explain dynamic distribution of interrupt requests among the available CPUs.
- * Explain the arbitration mechanism used for dynamic distribution of interrupt requests among the available CPUs.
- * State the purpose of TPR register in Local APIC.
- * What do you mean by IPI ?
- * How are inter-processor interrupts handled?
- * Describe the two distinct ways in which an I/O APIC may be configured in many of the current uniprocessor systems.
- * Classify the following exceptions to fault, trap or abort categories and write the interrupt type(vector) associated with each one of them.

Divide error, Debug, Breakpoint, Overflow, Bounds check, Invalid opcode, Device not available, Double fault, Coprocessor segment overrun, Invalid TSS, Segment not present, Stack segment, General protection, Page fault, Floating point error, Alignment check, Machine check, SIMD floating point exception

SOC 3010

OPERATING SYSTEM

END OF WEEK 5 LECTURE 2

**INTERRUPTS &
EXCEPTIONS**

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

